

PKCERT AI & Software Development Internship

Task 18 – Intro to PyTorch: Tensors, Autograd & Building a Simple Neural Network

Objective: Having implemented perceptrons, activation functions, and backpropagation manually from scratch in Task 16, interns will now transition to a high-level deep learning framework. This task builds practical fluency with PyTorch's tensor operations, its automatic differentiation engine (autograd), and the nn.Module API — while explicitly mapping each concept back to the manual derivations from Task 16 to reinforce the underlying mathematics.

Total Marks: 100

Part A – Tensors & Basic Operations (20 Marks)

- Install PyTorch and verify the installation (CPU is sufficient; GPU optional but may be used if available).
- Create tensors using at least four different methods (e.g., `torch.tensor`, `torch.zeros/torch.ones`, `torch.arange`, `torch.rand`, and conversion from a NumPy array) and explain when each is appropriate.
- Demonstrate tensor indexing, slicing, reshaping (view vs. reshape), and broadcasting rules with worked examples.
- Perform a moderately large matrix multiplication using both NumPy arrays and PyTorch tensors, time each, and briefly discuss any performance difference observed on your hardware.
- Explain, in your own words, the relationship between a `torch.Tensor` and a NumPy `ndarray`, including memory-sharing behavior when converting between them (`torch.from_numpy`, `.numpy()`).

Part B – Autograd (20 Marks)

- Explain what a computational graph is and how PyTorch constructs one dynamically (define-by-run) as operations are executed.
- Demonstrate `requires_grad`, `.backward()`, and `.grad` on a simple scalar expression, and manually verify that the computed gradient matches a hand-calculated derivative.
- Demonstrate gradient accumulation across multiple `.backward()` calls, and explain why `zero_grad()` must be called before each new training step.
- Explain the purpose of `torch.no_grad()` and `.detach()`, giving an example of when each is required (e.g., during evaluation, or when updating weights manually outside an optimizer).
- Using a small function of your choice, compare the gradients computed automatically by autograd against the manual backpropagation derivation from Task 16 Part C. Report the maximum numerical difference between the two and confirm they agree within a reasonable tolerance.

Part C – Building & Training a Simple Neural Network (45 Marks)

- Use the same dataset and architecture as your Task 16 Part C mini-project, so results can be meaningfully compared.
- Build a feed-forward neural network as a `torch.nn.Module` subclass (custom `__init__` and forward methods) with at least one hidden layer.

- Select and justify a loss function (e.g., `nn.CrossEntropyLoss`) and an optimizer (e.g., `torch.optim.SGD` or `torch.optim.Adam`).
- Implement the training loop explicitly — forward pass, loss computation, `loss.backward()`, `optimizer.step()`, `optimizer.zero_grad()` — tracking and plotting the loss curve over epochs.
- Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Compare this PyTorch implementation's performance and training time against your manual NumPy implementation from Task 16 and against the scikit-learn `MLPClassifier` baseline.
- Perform at least one hyperparameter experiment (learning rate, optimizer choice, or hidden layer size) and report the effect on training convergence and final performance.

Part D – Analysis & Documentation (15 Marks)

- Save the trained model's parameters using `torch.save (state_dict)` and demonstrate correctly reloading them with `load_state_dict`.
- Summarize the full pipeline: architecture, loss function, optimizer, and key hyperparameters chosen, with justification for each.
- Discuss at least two concrete differences observed between the manual NumPy implementation (Task 16) and this PyTorch implementation — e.g., in code complexity, training speed, numerical stability, or ease of experimentation.
- Reflect on what autograd abstracts away, and why the manual backpropagation exercise from Task 16 remains valuable even when using a framework.
- Describe one limitation or 'gotcha' encountered while learning PyTorch (e.g., shape mismatches, in-place operation errors, forgetting `zero_grad`, device mismatches) and how you resolved it.

Assessment Breakdown

Assessment Area	Marks
Tensors & Basic Operations	20
Autograd	20
Building & Training a Simple Neural Network	45
Analysis & Documentation	15
Total	100

Note: Unlike Task 16, the use of PyTorch's built-in autograd engine and `nn.Module` API is required and expected throughout this task — the intent is to build practical fluency with a production framework, in direct contrast to Task 16's from-scratch requirement. Document the PyTorch version used (e.g., "PyTorch 2.x") in your submission for reproducibility.